

# EEE3097/8/9S Micromouse 2026: Submission 4

## Final Maze Solver Code & Demo (25% of Course Mark)

---

### 1. Objective

Design and implement the complete autonomous intelligence for your Micromouse. The robot must explore a 4x6 grid maze to discover its wall layout, build a map, calculate the optimal shortest path back to the starting cell (or to the target cell), and execute a high-speed “solving run” without colliding with any walls.

#### Specific Learning Objectives:

- Calibrate distance thresholds for three VL53L0X ToF sensors.
  - Fuse high-resolution wheel encoders with the gyroscope yaw rate to track coordinate state  $(x, y)$  and heading direction.
  - Implement grid-based exploration state machines (e.g. Floodfill or DFS).
  - Implement shortest-path planning solvers (e.g., Dijkstra, A\*, or BFS) to calculate optimal routing.
  - Design velocity profiles to transition between straightaways and turns.
- 

### 2. Step-by-Step Implementation Guide

- **Exploration & Mapping:** Drive the mouse autonomously from  $(0, 0)$  to explore the grid. Keep the mouse centered using side ToF sensors.
  - **Path Solving:** Once the maze is mapped, calculate the shortest path to the target. Return to  $(0, 0)$  and stop.
  - **High-Speed Speed Run:** Load the calculated shortest path and sprint directly to the target. Merge consecutive straight cells into a single acceleration-cruise-deceleration profile. Stop autonomously inside the target.
- 

### 3. Deliverables (Gradescope Submission)

Run the packaging command from your repository root:

```
python tools/package_submission.py --task final_demo --src workspace/final_task/
```

Upload the resulting `submission_final_demo.zip` to Gradescope. The package must contain:

1. **Your Solving Code:** Python scripts recursively from your workspace OR your Simulink model `(.slx)` and generated C code directory `(*_ert_rtw/)`.
  2. **Physical Telemetry Log (`run_log.jsonl`):** Telemetry from your physical exploration and speed runs.
  3. **Physical Run Video (`run_video.mp4`):** An uncut video showing:
    - A clear **3-second close-up of your Student Card** at the start.
    - The complete mapping and high-speed solving run.
-

#### 4. Code & Log Check (Anti-Cheat)

- **Code Match Check:** The autograder compiles/hashes your code and verifies it matches the FNV-1a "hash" checksum in your telemetry header.
  - **Hardware ID Check:** Microcontroller Device UIDs are tracked. Shared logs under different student accounts will trigger a plagiarism audit.
- 

#### 5. Evaluation Criteria

- **Part A: Co-Simulation Speed & Accuracy (80%):** Graded proportionally on cells successfully visited and mapped, minus a –10% penalty for wall collisions. Speed run execution adds bonus speed traversal points.
- **Part B: Physical Log & Video Audit (20%):** Valid video with student card (10 points) and matched checksum log (10 points).